

Entendimiento de datos

Proyecto RaSA — Perfilamiento, análisis de calidad y conclusiones

Base de datos fuente: **RaSaTransaccional** · Tecnología: PySpark sobre JupyterLab. Documento con el **código** y su **ejecución** (salidas reales).

Resumen ejecutivo

El trabajo comprende un diagnóstico de las fuentes (perfilamiento y análisis de calidad), la limpieza y depuración de los datos, y las conclusiones sobre el conjunto ya depurado. Las fuentes contienen la información necesaria para relacionar planes, beneficios, áreas de servicio y condiciones de pago; tras la depuración quedan en condiciones de integrarse mediante un esquema estrella y de soportar los tableros de control y análisis solicitados por RaSA.

Indicadores clave del diagnóstico inicial (datos sin depurar)

Fuente	Registros	Columnas	Duplicados completos	Hallazgo principal (previo a la limpieza)
Áreas de servicio	188.815	9	43.573 (23,1%)	Fechas y áreas fuera de rango; llave con nulos
Tipos de beneficio	849	9	271 (31,9%)	Banderas en varios idiomas; 178 ≠ 170 esperados
Planes-beneficio	36.036	11	8.493 (23,6%)	Llaves foráneas nulas y huérfanas; grano indefinido
Condiciones de pago	31	3	7 (22,6%)	Identificador repetido; 14 copagos (esperado 15)

Proyecto RaSA — Entendimiento de datos

Entregable 1 — Perfilamiento de datos

Se exploran las cuatro fuentes compartidas por RaSA en la base [RaSaTransaccional](#) para entender su estructura, volumen y contenido, y dejar listas las observaciones que alimentan el análisis de calidad (Entregable 2) y las conclusiones de viabilidad (Entregable 3).

Fuentes: - **F1. FuenteAreasDeServicio_Copia_E** — áreas de servicio por condado. - **F2. FuenteTiposBeneficio_Copia_E** — tipos de beneficio. - **F3. FuentePlanesBeneficio_Copia_E** — beneficios ofrecidos por cada plan (tabla central). - **F4. FuenteCondicionesDePago_Copia_E** — condiciones de pago (referencia).

0. Configuración y utilidades

In

```
# Credenciales y conexión (use su usuario/clave del curso y su servidor de los tutoriales)
db_user = 'DB_202613_f_cucina'
db_psswd = 'REDACTADO' # complete con su contraseña del curso (no se publica en el PDF)

# Base de datos de las fuentes
source_db_connection_string = 'jdbc:mysql://157.253.236.120:8080/RaSaTransaccional' # AJUSTE host/puerto al suyo

# Driver JDBC de MySQL
path_jar_driver = 'drivers/mysql-connector-j-8.0.33.jar'
```

In

```
import os
from pyspark.sql import SparkSession, functions as F

# Configuración de Spark para usar el driver JDBC

os.environ["SPARK_LOCAL_IP"] = "127.0.0.1"
os.environ["SPARK_LOCAL_HOSTNAME"] = "localhost"

if not os.path.exists(path_jar_driver):
    raise FileNotFoundError(f'No se encontró el driver JDBC en: {path_jar_driver}')

spark = (
    SparkSession.builder
        .appName("CalidadDBdatos_RaSA")
        .master("local[*]")
        .config("spark.driver.host", "127.0.0.1")
        .config("spark.driver.bindAddress", "127.0.0.1")
        .config("spark.jars", path_jar_driver)
        .config("spark.driver.extraClassPath", path_jar_driver)
        .config("spark.executor.extraClassPath", path_jar_driver)
        .getOrCreate()
)

spark.conf.set("spark.sql.legacy.timeParserPolicy", "LEGACY")
```

In

```
def obtener_dataframe_de_bd(db_connection_string, sql, db_user, db_psswd):
    """
    Esta función obtiene un DataFrame de Spark a partir de una consulta SQL a una base de datos MySQL.
    """
    return spark.read.format('jdbc') \
        .option('url', db_connection_string) \
        .option('dbtable', sql) \
        .option('user', db_user) \
        .option('password', db_psswd) \
        .option('driver', 'com.mysql.cj.jdbc.Driver') \
        .load()

def cargar_tabla(nombre):
    """
    Esta función carga una tabla de la base de datos RaSaTransaccional y devuelve un DataFrame de Spark.
    """
    sql = f'(SELECT * FROM RaSaTransaccional.{nombre}) AS t'
    return obtener_dataframe_de_bd(source_db_connection_string, sql, db_user, db_psswd)
```

In

```
# --- Funciones de perfilamiento reutilizables ---
def resumen(df, nombre):
    """
    Esta función muestra un resumen del DataFrame, incluyendo el número de filas, columnas y el esquema de datos.
    """
    print(f'== {nombre} ==')
    print('Filas:', df.count(), '| Columnas:', len(df.columns))
    df.printSchema()

def nulos(df):
    """
    Esta función muestra el número de valores nulos en cada columna del DataFrame.
    """
    total = df.count()
    fila = df.select([F.count(F.when(F.col(c).isNull(), c)).alias(c) for c in df.columns]).collect()[0]
    print(f'Total filas: {total}')
    for c in df.columns:
        n = fila[c]
        print(f' {c}: {n} nulos ({100*n/total:.1f}%)')

def cardinalidad(df):
    """
    Esta función muestra el número de valores distintos en cada columna del DataFrame.
    """
    fila = df.select([F.countDistinct(F.col(c)).alias(c) for c in df.columns]).collect()[0]
    for c in df.columns:
        print(f' {c}: {fila[c]} valores distintos')

def frecuencias(df, columna, n=20):
    """
    Esta función muestra las frecuencias de los valores en una columna del DataFrame.
    """
    df.groupBy(columna).count().orderBy('count', ascending=False).show(n, truncate=False)

def anio(colname='Fecha'):
    """
    Esta función extrae el año de una columna de fechas.
    """
    # Extrae el año como texto (robusto aunque la fecha venga como string)
    return F.substring(F.col(colname).cast('string'), 1, 4)
```

In

```
# Verifique los nombres reales de las fuentes
tablas_bd = obtener_dataframe_de_bd(source_db_connection_string,
    "(SELECT table_name FROM information_schema.tables WHERE table_schema='RaSaTransaccional' AND table_name LIKE 'Fuente%') AS t",
    db_user, db_psswd)
tablas_bd.show(truncate=False)

TABLAS = {
    'areas': 'FuenteAreasDeServicio Copia E',
    'tipos': 'FuenteTiposBeneficio Copia E',
    'planes': 'FuentePlanesBeneficio Copia E', # ajuste si en la BD es 'FuentesPlanes...'
    'condiciones': 'FuenteCondicionesDePago_Copia_E',
}
```

Out

```
+-----+
|TABLE_NAME|
+-----+
|FuenteAreasDeServicio Copia E|
|FuenteCondicionesDePago_Copia E|
|FuentePlanesBeneficio Copia E|
|FuenteTiposBeneficio_Copia E|
+-----+
```

F1 — FuenteAreasDeServicio_Copia_E

Significado de una fila: un área de servicio definida sobre un condado (con su población, área y densidad). El negocio indica que deben ser **5409 áreas** y que cubren todos los condados del país.

In

```
areas = cargar_tabla(TABLAS['areas'])
resumen(areas, 'Áreas de servicio')
```

Out

```
=== Áreas de servicio ===
Filas: 188815 | Columnas: 9
root
|-- IDAreaDeServicio.T: integer (nullable = true)
|-- NombreAreaDeServicio: string (nullable = true)
|-- IDGeografia.T: integer (nullable = true)
|-- Condado: string (nullable = true)
|-- Estado: string (nullable = true)
|-- PoblacionAct: float (nullable = true)
|-- Area: float (nullable = true)
|-- Densidad: float (nullable = true)
|-- Fecha: integer (nullable = true)
```

Conteo de nulos por columna (completitud) y cardinalidad (¿es `IdAreaDeServicio_T` única?).

In	<pre>print('--- Nulos ---'); nulos(areas) print('--- Valores distintos ---'); cardinalidad(areas)</pre>
Out	<pre>--- Nulos --- Total filas: 188815 IdAreaDeServicio T: 6288 nulos (3.3%) NombreAreaDeServicio: 0 nulos (0.0%) IdGeografia T: 6378 nulos (3.4%) Condado: 0 nulos (0.0%) Estado: 0 nulos (0.0%) PoblacionAct: 0 nulos (0.0%) Area: 2407 nulos (1.3%) Densidad: 2407 nulos (1.3%) Fecha: 0 nulos (0.0%) --- Valores distintos --- IdAreaDeServicio T: 5409 valores distintos NombreAreaDeServicio: 5412 valores distintos IdGeografia T: 4251 valores distintos Condado: 1398 valores distintos Estado: 35 valores distintos PoblacionAct: 3729 valores distintos Area: 1957 valores distintos Densidad: 495 valores distintos Fecha: 3 valores distintos</pre>

Estadísticos de las columnas numéricas y revisión de las cifras del negocio.

In	<pre>areas.select('PoblacionAct', 'Area', 'Densidad').describe().show() areas_distintas = areas.select(F.countDistinct('IdAreaDeServicio_T').alias('n')).collect()[0]['n'] condados_distintos = areas.select(F.countDistinct('IdGeografia_T').alias('n')).collect()[0]['n'] print('Áreas distintas (esperado 5409):', areas_distintas) print('Condados distintos:', condados_distintos) print('--- Estados ---'); frecuencias(areas, 'Estado') print('--- Años (Fecha) ---'); areas.select(anoio().alias('anio')).groupBy('anio').count().orderBy('anio').show()</pre>
Out	<pre>+-----+-----+-----+ summary PoblacionAct Area Densidad +-----+-----+-----+ count 188815 186408 186408 mean 4.147959039464555E7 770.9518851122269 276.0979410754903 stddev 5.943835891771832E9 1263.4807340485252 851.3219978151402 min 82.0 -24707.0 0.0 max 4.72803E10 88824.0 14946.0 +-----+-----+-----+ Áreas distintas (esperado 5409): 5409 Condados distintos: 4251 --- Estados --- +-----+-----+ Estado count +-----+-----+ Indiana 14159 Missouri 13913 Georgia 13822 Wisconsin 12543 Michigan 11598 Texas 11261 South Dakota 10930 Kansas 10824 Kentucky 10571 Oklahoma 8183 Virginia 7553 Florida 7440 Mississippi 7234 Illinois 5965 Montana 5789 North Carolina 5589 New Jersey 4990 West Virginia 4613 Ohio 4033 Pennsylvania 3240 +-----+-----+ only showing top 20 rows --- Años (Fecha) --- +-----+-----+ anio count +-----+-----+ 1800 6282 2017 99211 2018 83322 +-----+-----+</pre>

Observaciones — Áreas de servicio

- **Volumen y unicidad:** la tabla tiene **188.815 filas** pero solo **5.409 `IdAreaDeServicio_T` distintos** — justo la cifra que reporta el negocio. Es decir, cada área aparece repetida ~35 veces: el identificador del área **no es único por fila** (fuerte duplicación). `NombreAreaDeServicio` tiene 5.412 distintos (3 más que ids), lo que sugiere pequeñas variaciones de nombre para una misma área.
- **Completitud:** la llave `IdAreaDeServicio_T` tiene **3,3% de nulos** (6.288 filas) e `IdGeografia_T` **3,4%** (6.378); `Area` y `Densidad` **1,3%** (2.407). Que la llave del área venga nula es crítico porque impide integrar esas filas.
- **Validez:** `Area` tiene un mínimo **negativo (−24.707)**, imposible para un área; `PoblacionAct` llega a $\approx 4,7 \times 10^{10}$ (**47 mil millones**) con una desviación enorme → valores atípicos/corruptos. `Densidad` mínimo 0.
- **Consistencia temporal:** `Fecha` toma 3 años: 2017 (99.211), 2018 (83.322) y **1800 (6.282)**, que es claramente inválido. Además **no aparece 2019**, aunque el negocio indica que la información va de 2017 a 2019.
- **Cobertura:** solo hay **35 estados** (de ~50) y cerca de **4,25 mil condados**. Según el método de conteo usado, se observan **4.251 condados no nulos** o **4.252 valores distintos** cuando se incluye la categoría nula en el conteo.

F2 — FuenteTiposBeneficio_Copia_E

Significado de una fila: un tipo de beneficio y sus atributos (si es EHB, si tiene límite cuantitativo, exclusiones, etc.). El negocio indica **170 tipos de beneficio**.

In	<pre>tipos = cargar_tabla(TABLAS['tipos']) resumen(tipos, 'Tipos de beneficio')</pre>
Out	<pre>=== Tipos de beneficio === Filas: 849 Columnas: 9 root -- IdTipoBeneficio T: integer (nullable = true) -- Nombre: string (nullable = true) -- UnidadDelLmite: string (nullable = true) -- EsEHB: string (nullable = true) -- EstaCubiertaPorSeguro: string (nullable = true) -- TieneLmiteCuantitativo: string (nullable = true) -- ExcluidoDelDesembolsoMaximoDentroDeLaRed: string (nullable = true) -- ExcluidoDelDesembolsoMaximoFueraDeLaRed: string (nullable = true) -- Fecha: integer (nullable = true)</pre>
In	<pre>print('--- Nulos ---'); nulos(tipos) print('--- Valores distintos ---'); cardinalidad(tipos) print('Tipos distintos (esperado 170):', tipos.select('IdTipoBeneficio_T').distinct().count())</pre>
Out	<pre>--- Nulos --- Total filas: 849 IdTipoBeneficio T: 0 nulos (0.0%) Nombre: 0 nulos (0.0%) UnidadDelLmite: 0 nulos (0.0%) EsEHB: 0 nulos (0.0%) EstaCubiertaPorSeguro: 0 nulos (0.0%) TieneLmiteCuantitativo: 0 nulos (0.0%) ExcluidoDelDesembolsoMaximoDentroDeLaRed: 0 nulos (0.0%) ExcluidoDelDesembolsoMaximoFueraDeLaRed: 0 nulos (0.0%) Fecha: 0 nulos (0.0%) --- Valores distintos --- IdTipoBeneficio T: 178 valores distintos Nombre: 178 valores distintos UnidadDelLmite: 63 valores distintos EsEHB: 3 valores distintos EstaCubiertaPorSeguro: 3 valores distintos TieneLmiteCuantitativo: 4 valores distintos ExcluidoDelDesembolsoMaximoDentroDeLaRed: 3 valores distintos ExcluidoDelDesembolsoMaximoFueraDeLaRed: 2 valores distintos Fecha: 2 valores distintos Tipos distintos (esperado 170): 178</pre>

Distribución de las banderas (booleanas) y de la unidad del límite.

In	<pre>for c in ['EsEHB', 'EstaCubiertaPorSeguro', 'TieneLmiteCuantitativo', 'ExcluidoDelDesembolsoMaximoDentroDeLaRed', 'ExcluidoDelDesembolsoMaximoFueraDeLaRed']: print(f'--- {c} ---'); frecuencias(tipos, c) print('--- UnidadDelLmite ---'); frecuencias(tipos, 'UnidadDelLmite')</pre>
Out	<pre>--- EsEHB --- +-----+-----+ EsEHB count +-----+-----+ Yes 430 </pre>

```
|No |399 |
|True |20 |
+-----+
--- EstaCubiertaPorSeguro ---
+-----+
|EstaCubiertaPorSeguro|count|
+-----+
|Yes |743 |
|No |98 |
|False |8 |
+-----+
--- TieneLimiteCuantitativo ---
+-----+
|TieneLimiteCuantitativo|count|
+-----+
|No |542 |
|Yes |271 |
|Nein |24 |
|Si |12 |
+-----+
--- ExcluidoDelDesembolsoMaximoDentroDeLaRed ---
+-----+
|ExcluidoDelDesembolsoMaximoDentroDeLaRed|count|
+-----+
|No |787 |
|Yes |60 |
|Algunas veces |2 |
+-----+
--- ExcluidoDelDesembolsoMaximoFueraDeLaRed ---
+-----+
|ExcluidoDelDesembolsoMaximoFueraDeLaRed|count|
+-----+
|No |516 |
|Yes |333 |
+-----+
--- UnidadDelLimite ---
+-----+
|UnidadDelLimite |count|
+-----+
| |559 |
|Visit(s) per Year |64 |
|Visit(s) per Benefit Period |31 |
|Days per Year |14 |
|Days per Benefit Period |13 |
|Visit(s) per YearVisit(s) per YearVisit(s) per Year |11 |
|Item(s) per Year |10 |
|Exam(s) per Year |10 |
|Dollars per Year |9 |
|Visit(s) per Benefit PeriodVisit(s) per Benefit PeriodVisit(s) per Benefit Period |7 |
|Visit(s) per 6 Months |7 |
|Visit(s) per year |6 |
|Item(s) per Benefit Period |6 |
|Procedure(s) per Year |6 |
|Procedure(s) per Lifetime |5 |
|Procedure(s) per 3 Years |5 |
|Treatment(s) per Lifetime |4 |
|Exam(s) per Benefit Period |4 |
|Item(s) per 2 Years |4 |
|Procedure(s) per Benefit Period |4 |
+-----+
only showing top 20 rows
```

Observaciones — Tipos de beneficio

- **Compleitud: 0 nulos** en todas las columnas (buena completitud).
- **Unicidad: 849 filas** con solo **178 `IdTipoBeneficio_T` distintos** → cada tipo se repite ~5 veces (no es único por fila). Además **178 ≠ 170** esperados: hay 8 tipos de más, posiblemente espurios o duplicados con variación.
- **Validez (banderas con codificación inconsistente y mezcla de idiomas):**
- **EsEHB** : Yes (430) / No (399) y además **True (20)**.
- **EstaCubiertaPorSeguro** : Yes (743) / No (98) y además **False (8)**.
- **TieneLimiteCuantitativo** : No (542) / Yes (271) y además **Nein (24, alemán)** y **Si (12, español)** → 4 valores en 3 idiomas.
- **ExcluidoDelDesembolsoMaximoDentroDeLaRed** : No (787) / Yes (60) y además **Algunas veces (2)**.
- **UnidadDelLimite** : **559 vacíos** y cadenas corruptas/**triplicadas** (p. ej. "Visit(s) per YearVisit(s) per YearVisit(s) per Year").
- Estas banderas deben **estandarizarse a un único formato** (p. ej. Yes/No) antes de cualquier análisis.

F3 — FuentePlanesBeneficio_Copia_E

Significado de una fila: un beneficio concreto ofrecido por un plan, en un área de servicio, con sus valores de copago/coseguro y condiciones. Es la **tabla central** que relaciona todas las demás. El negocio indica **301 planes en 2017 y 422 en 2018**, y para 2018 los máximos de copago/coseguro son **3300 y 100**.

In

planes = cargar_tabla(TABLES['planes'])
resumen(planes, 'Planes-beneficio')

Out

=== Planes-beneficio ===
Filas: 36036 | Columnas: 11
root
|-- IdTipoBeneficio.T: long (nullable = true)
|-- IdAreaDeServicio.T: long (nullable = true)
|-- IdCondicionDePagoCopago.T: integer (nullable = true)
|-- IdCondicionDePagoCoseguro.T: integer (nullable = true)
|-- IdNivelServicio.T: integer (nullable = true)
|-- IdPlan.T: string (nullable = true)
|-- Fecha: string (nullable = true)
|-- IdProveedor.T: integer (nullable = true)
|-- valorCopago: integer (nullable = true)
|-- valorCoseguro: integer (nullable = true)
|-- cantidadLimite: integer (nullable = true)

In

print('--- Nulos ---'); nulos(planes)
print('--- Valores distintos ---'); cardinalidad(planes)

Out

-- Nulos --
Total filas: 36036
IdTipoBeneficio.T: 2086 nulos (5.8%)
IdAreaDeServicio.T: 2041 nulos (5.7%)
IdCondicionDePagoCopago.T: 0 nulos (0.0%)
IdCondicionDePagoCoseguro.T: 0 nulos (0.0%)
IdNivelServicio.T: 0 nulos (0.0%)
IdPlan.T: 0 nulos (0.0%)
Fecha: 0 nulos (0.0%)
IdProveedor.T: 0 nulos (0.0%)
valorCopago: 0 nulos (0.0%)
valorCoseguro: 0 nulos (0.0%)
cantidadLimite: 30571 nulos (84.8%)
-- Valores distintos --
IdTipoBeneficio.T: 285 valores distintos
IdAreaDeServicio.T: 6496 valores distintos
IdCondicionDePagoCopago.T: 34 valores distintos
IdCondicionDePagoCoseguro.T: 5 valores distintos
IdNivelServicio.T: 3 valores distintos
IdPlan.T: 393 valores distintos
Fecha: 5 valores distintos
IdProveedor.T: 125 valores distintos
valorCopago: 49 valores distintos
valorCoseguro: 25 valores distintos
cantidadLimite: 41 valores distintos

Planes distintos por año (validación 301 en 2017, 422 en 2018) y estadísticos de los valores.

In

planes.withColumn('anio', anio()).groupBy('anio') \
 .agg(F.countDistinct("IdPlan_T").alias('planes_distintos')) \
 .orderBy('anio').show()

planes.select('valorCopago', 'valorCoseguro', 'cantidadLimite').describe().show()

print('--- Máximos copago/coseguro 2018 (esperado 3300 y 100) ---')
planes.withColumn('anio', anio()).filter(F.col('anio') == '2018') \
 .select(F.max('valorCopago').alias('max_copago'),
 F.max('valorCoseguro').alias('max_coseguro')).show()

Out

+---+-----+
|anio|planes distintos|
+---+-----+
1920	212
2017	203
2018	286
Dec	218
+---+-----+

+-----+-----+-----+-----+-----+
|summary| valorCopago| valorCoseguro| cantidadLimite|
+-----+-----+-----+-----+-----+
count	36036	36036	5465
mean	9.778416028416029	24.29279854279054	129.3313815187557
stddev	100.41137967781772	36.959755494458506	951.2795862336690
min	0	0	1
max	3500	100	30000
+-----+-----+-----+-----+-----+

```
--- Máximos Copago/coseguro 2018 (esperado 3300 y 100) ---
+-----+-----+
|max_copago|max_coseguro|
+-----+-----+
|      3300|      100|
+-----+-----+
```

Observaciones — Planes-beneficio

- **Compleitud:** `IdTipoBeneficio_T` **5,8% nulos** (2.086) e `IdAreaDeServicio_T` **5,7%** (2.041) → llaves foráneas incompletas. `cantidadLimite` está **84,8% nula** (30.571): es esperable porque solo aplica a los tipos con límite cuantitativo; los valores no nulos van de **1 a 30.000 (sin ceros)**, lo cual es coherente con la regla del negocio.
- **Validez:** `valorCopago` llega a **3.500** y el máximo de **2018 es 3.500**, mientras el negocio indica un tope de **3.300** → excede en 200 (posible error/outlier). `valorCoseguro` máximo **100** (coincide con el negocio).
- **Conteo de planes:** hay **393** `IdPlan_T` distintos, pero el conteo por año **no es confiable** porque `Fecha` viene en **formatos mezclados** (aparecen "1920", "Dec ..." además de 2017/2018). Por eso no se obtienen los 301/422 esperados: primero hay que **estandarizar la fecha**.
- **Integridad referencial (consistencia):** los planes referencian **285** tipos de beneficio y **6.496** áreas, pero las fuentes respectivas solo tienen **178** tipos y **5.409** áreas → existen llaves foráneas que **apuntan a registros inexistentes**. `IdCondicionDePagoCopago_T` tiene 14 distintos (negocio: 15) y `IdCondicionDePagoCoseguro_T` 5 (coincide).

F4 — FuenteCondicionesDePago_Copia_E

Significado de una fila: una condición de pago (referencia). En el perfilamiento se observan principalmente categorías de copago y coseguro; adicionalmente aparecen valores no estandarizados en el campo `Tipo` que deben revisarse en el análisis de calidad.

In

condiciones = cargar_tabla(TABLAS['condiciones'])
resumen(condiciones, 'Condiciones de pago')

Out

=== Condiciones de pago ===
Filas: 31 | Columnas: 3
root
|-- IdCondicionesDePago.T: integer (nullable = true)
|-- Description: string (nullable = true)
|-- Tipo: string (nullable = true)

In

print('--- Nulos ---'); nulos(condiciones)
print('--- Valores distintos ---'); cardinalidad(condiciones)
print('--- Tipo (esperado: 15 Copago, 5 Coseguro) ---'); frecuencias(condiciones, 'Tipo')
print('--- Description ---'); frecuencias(condiciones, 'Description')

Out

--- Nulos ---
Total filas: 31
IdCondicionesDePago.T: 0 nulos (0.0%)
Description: 0 nulos (0.0%)
Tipo: 0 nulos (0.0%)
--- Valores distintos ---
IdCondicionesDePago.T: 23 valores distintos
Description: 17 valores distintos
Tipo: 6 valores distintos
--- Tipo (esperado: 15 Copago, 5 Coseguro) ---
+-----+-----+
|Tipo|count|
+-----+-----+
Copago	19
Coseguro	17
NaN	2
Copagado	1
Coseguridad	1
SinTipoCopago	1
+-----+-----+	
--- Description ---	
+-----+-----+	
Description	count
+-----+-----+	
Copay per Stay	3
No Charge after deductible	3
No Charge	3
Copay per Stay after deductible	3
Coinsurance	3
Coinsurance after deductible	2
Copay per Day with deductible	2
Copay	2
Not Applicable	2
Copay before deductible	1
Copay per Stay with deductible	1
Copay per Day after deductible	1
Copay with deductible	1
Copay after deductible	1
Copay per Day before deductible	1
Copay per Stay before deductible	1
Copay per Day	1
+-----+-----+

Observaciones — Condiciones de pago

- **Compleitud:** no se observan valores nulos en las columnas de la fuente.
- **Unicidad:** se identifican repeticiones del identificador `IdCondicionesDePago_T` y de la descripción, por lo que la fila no representa una entidad única por sí sola.
- **Dominio observado en Tipo** : predominan categorías de copago y coseguro, y también aparecen valores no estandarizados (por ejemplo, variantes ortográficas y categorías no esperadas). Estos resultados se toman como insumo para la validación formal de dominios en el Entregable 2.

Relaciones entre fuentes (insumo para el Entregable 3)

La tabla central `FuentePlanesBeneficio_Copia_E` integra las demás mediante las siguientes llaves observadas:

Desde (Planes)	Hacia	Columna destino
<code>IdTipoBeneficio_T</code>	<code>FuenteTiposBeneficio_Copia_E</code>	<code>IdTipoBeneficio_T</code>
<code>IdAreaDeServicio_T</code>	<code>FuenteAreasDeServicio_Copia_E</code>	<code>IdAreaDeServicio_T</code>
<code>IdCondicionDePagoCopago_T</code>	<code>FuenteCondicionesDePago_Copia_E</code>	<code>IdCondicionesDePago_T</code> (rol Copago)
<code>IdCondicionDePagoCoseguro_T</code>	<code>FuenteCondicionesDePago_Copia_E</code>	<code>IdCondicionesDePago_T</code> (rol Coseguro)

Además, la columna `IdGeografia_T` de `FuenteAreasDeServicio_Copia_E` identifica la geografía (condado). La validación formal de integridad referencial y consistencia entre fuentes se desarrolla en los entregables de calidad y conclusiones.

Síntesis del perfilamiento (insumo para los Entregables 2 y 3)

Fuente	Volumen y estructura observada	Hallazgos descriptivos del perfilamiento	Implicación para siguientes entregables
Áreas de servicio	188.815 registros; 9 columnas; identificador de área presente en alta proporción	Se observan repeticiones del identificador de área, valores faltantes en campos clave, valores atípicos en variables numéricas y registros de fecha fuera del rango esperado	Entregable 2: evaluar formalmente completitud, unicidad y validez. Entregable 3: validar impacto en análisis de cobertura y concentración
Tipos de beneficio	849 registros; 9 columnas; catálogo con atributos de cobertura y límite	Se observan repeticiones del identificador y heterogeneidad en los valores de campos tipo bandera y categóricos	Entregable 2: formalizar reglas de dominio y unicidad. Entregable 3: revisar impacto en análisis por tipo de beneficio
Planes-beneficio	36.036 registros; 11 columnas; tabla central de integración	Se identifican valores faltantes en campos de relación, formatos heterogéneos en fecha y dispersión en variables de costo/límite	Entregable 2: validar completitud, validez y consistencia con reglas de negocio. Entregable 3: confirmar viabilidad de integración entre fuentes
Condiciones de pago	31 registros; 3 columnas; tabla de referencia	Se observan repeticiones del identificador y variaciones en los valores del campo <code>Tipo</code>	Entregable 2: validar dominio y unicidad. Entregable 3: confirmar uso como dimensión de referencia

El perfilamiento evidencia que las fuentes contienen información útil para los análisis solicitados, pero también patrones que requieren evaluación formal de calidad en el Entregable 2. Por ello, este entregable se mantiene como diagnóstico exploratorio y trazabilidad de hallazgos, mientras que la determinación de cumplimiento frente a reglas de negocio se desarrolla en calidad de datos.

Limitaciones del perfilamiento

- Los resultados dependen del estado observado de las fuentes en el momento de la extracción.
- En algunas columnas de fecha se evidencian formatos heterogéneos, por lo que los conteos por año son exploratorios hasta su estandarización.
- La presencia de nulos y repeticiones en identificadores puede afectar la lectura directa de cardinalidades y relaciones.
- La validación definitiva contra reglas de negocio se realiza en el Entregable 2 (calidad) y su impacto analítico se consolida en el Entregable 3.

Proyecto RaSA — Entendimiento de datos 2

Entregable 2 — Calidad de datos

Se revisan las 4 dimensiones de calidad vistas en el curso en las fuentes compartidas por RaSA en la base `RaSaTransaccional` y se presentan las conclusiones del análisis. - **1. Completitud** - **2. Unicidad** - **3. Validez** - **4. Consistencia**

Las fuentes evaluadas son:

- **F1. Fuente**`AreasDeServicio_Copia_E` — áreas de servicio por condado.
- **F2. Fuente**`TiposBeneficio_Copia_E` — tipos de beneficio.
- **F3. Fuente**`PlanesBeneficio_Copia_E` — beneficios ofrecidos por cada plan (tabla central).
- **F4. Fuente**`CondicionesDePago_Copia_E` — condiciones de pago (referencia).

La conexión, las funciones de carga y las tablas ya fueron definidas en el Entregable 1; a continuación se reutilizan directamente para el análisis de calidad.

1. Completitud

La completitud evalúa si los campos requeridos para los análisis de RaSA contienen información suficiente o si presentan valores nulos o vacíos que puedan impedir la integración de fuentes o el cálculo de indicadores.

In

```
from pyspark.sql import functions as F
from functools import reduce

fuentes = {
    "AreasDeServicio": areas,
    "TiposBeneficio": tipos,
    "PlanesBeneficio": planes,
    "CondicionesDePago": condiciones
}

def evaluar_completitud(df, nombre_fuente):
    """
    Esta función evalúa la completitud de los datos en un DataFrame de Spark,
    calculando el número total de registros, el número de valores faltantes y
    el porcentaje de valores faltantes para cada columna. Devuelve un nuevo DataFrame con los resultados.
    """
    total_registros = df.count()
    resultados = []

    for columna in df.columns:
        valores_faltantes = df.filter(
            F.col(columna).isNull() |
            (F.trim(F.col(columna).cast("string")) == "")
        ).count()

        porcentaje_faltantes = (
            round((valores_faltantes / total_registros) * 100, 2)
            if total_registros > 0 else 0
        )

        resultados.append((
            nombre_fuente,
            columna,
            total_registros,
            valores_faltantes,
            porcentaje_faltantes
        ))

    return spark.createDataFrame(
        resultados,
        ["Fuente", "Campo", "Total_Registros", "Valores_Faltantes", "Porcentaje_Faltantes"]
    )
```

In

```
resultados_completitud = []

for nombre_fuente, df in fuentes.items():
    resultados_completitud.append(
        evaluar_completitud(df, nombre_fuente)
    )

completitud = reduce(
    lambda df1, df2: df1.unionByName(df2),
    resultados_completitud
)
```

In

```
completitud_problemas = (
    completitud
    .filter(F.col("Valores_Faltantes") > 0)
    .orderBy(F.desc("Porcentaje_Faltantes"))
)

completitud_problemas.show(100, truncate=False)
```

Out

Fuente	Campo	Total_Registros	Valores_Faltantes	Porcentaje_Faltantes
PlanesBeneficio	cantidadLimite	36036	30571	84.83
TiposBeneficio	UnidadDelLimite	849	559	65.84
PlanesBeneficio	IdTipoBeneficio_T	36036	2086	5.79
PlanesBeneficio	IdPlan_T	36036	2053	5.7
PlanesBeneficio	IdAreaDeServicio_T	36036	2041	5.66
AreasDeServicio	IdGeografia_T	188815	6378	3.38
AreasDeServicio	IdAreaDeServicio_T	188815	6288	3.33
AreasDeServicio	Area	188815	2407	1.27
AreasDeServicio	Densidad	188815	2407	1.27

In

```
resumen_completitud = (
    completitud
    .groupBy("Fuente")
    .agg(
        F.count("Campo").alias("Total_Campos"),
        F.sum(F.when(F.col("Valores_Faltantes") > 0, 1).otherwise(0)).alias("Campos_Con_Faltantes"),
        F.sum("Valores_Faltantes").alias("Total_Valores_Faltantes"),
        F.max("Porcentaje_Faltantes").alias("Mayor_Porcentaje_Faltantes")
    )
    .orderBy("Fuente")
)

resumen_completitud.show(truncate=False)
```

Out

Fuente	Total_Campos	Campos_Con_Faltantes	Total_Valores_Faltantes	Mayor_Porcentaje_Faltantes
AreasDeServicio	19	4	17400	3.38
CondicionesDePago	3	0	0	0.0
PlanesBeneficio	11	4	36751	84.83
TiposBeneficio	19	1	559	65.84

Análisis de completitud

El análisis de completitud confirma los hallazgos identificados durante el perfilamiento inicial y permite evaluar su impacto sobre la calidad de los datos. Aunque algunas fuentes presentan un buen nivel de completitud, existen campos faltantes que afectan directamente la integración entre tablas y la construcción de análisis confiables.

La fuente `CondicionesDePago` no presenta valores faltantes en los campos evaluados, por lo que no se identifican problemas de completitud en esta fuente. En contraste, `PlanesBeneficio` concentra el mayor volumen de faltantes, principalmente en `cantidadLimite`, con 30.571 registros sin información, equivalentes al 84,83%. Este hallazgo debe analizarse junto con la regla de negocio de beneficios con límite cuantitativo, ya que no todos los beneficios necesariamente requieren una cantidad límite.

En `TiposBeneficio`, el campo `UnidadDelLimite` presenta 559 valores faltantes, equivalentes al 65,84%. Este resultado también debe interpretarse en relación con el atributo `TieneLimiteCuantitativo`, dado que la unidad del límite solo sería obligatoria para beneficios que manejan límites cuantitativos.

Por su parte, `AreasDeServicio` presenta faltantes en `IdGeografia_T`, `IdAreaDeServicio_T`, `Area` y `Densidad`. Aunque los porcentajes son bajos, entre 1,27% y 3,38%, el faltante en `IdAreaDeServicio_T` es especialmente crítico porque este campo permite relacionar las áreas de servicio con la fuente central de planes-beneficio.

En conclusión, la completitud de los datos es parcialmente adecuada para continuar con el proyecto. Sin embargo, antes de construir el modelo dimensional deben definirse reglas para tratar los campos faltantes, especialmente aquellos asociados con claves de integración, variables geográficas y atributos requeridos para validar límites cuantitativos.

2. Unicidad

La unicidad evalúa si los registros o identificadores permiten reconocer de manera única cada entidad dentro de una fuente. En esta sección se revisa la cantidad de registros totales, registros completamente

únicos, registros duplicados y valores distintos de las llaves principales identificadas en cada fuente.

Este análisis permite identificar posibles duplicidades, presencia de información histórica o problemas en la definición de llaves operacionales.

```
In def evaluar_unicidad(df, nombre_fuente, llave):  
    """  
    Esta función evalúa la unicidad de los datos en un DataFrame de Spark,  
    calculando el número total de registros, el número de registros únicos  
    y el número de duplicados completos para una columna específica. Devuelve un nuevo DataFrame con los resultados.  
    """  
    total_registros = df.count()  
    registros_unicos = df.distinct().count()  
    duplicados_completos = total_registros - registros_unicos  
  
    valores_distintos_llave = df.select(llave).distinct().count()  
  
    llaves_repetidas = (  
        df  
        .groupBy(llave)  
        .count()  
        .filter(F.col("count") > 1)  
        .count()  
    )  
  
    porcentaje_duplicados_completos = (  
        round((duplicados_completos / total_registros) * 100, 2)  
        if total_registros > 0 else 0  
    )  
  
    return spark.createDataFrame(  
        [(  
            nombre_fuente,  
            llave,  
            total_registros,  
            registros_unicos,  
            duplicados_completos,  
            porcentaje_duplicados_completos,  
            valores_distintos_llave,  
            llaves_repetidas  
        )],  
        [  
            "Fuente",  
            "Llave Evaluada",  
            "Total Registros",  
            "Registros Unicos",  
            "Duplicados Completos",  
            "Porcentaje Duplicados Completos",  
            "Valores Distintos Llave",  
            "Llaves Repetidas"  
        ]  
    )
```

```
In unidad_areas = evaluar_unicidad(  
    areas,  
    "AreasDeServicio",  
    "IdAreaDeServicio_T"  
)  
  
unidad_tipos = evaluar_unicidad(  
    tipos,  
    "TiposBeneficio",  
    "IdTipoBeneficio_T"  
)  
  
unidad_planes = evaluar_unicidad(  
    planes,  
    "PlanesBeneficio",  
    "IdPlan_T"  
)  
  
unidad_condiciones = evaluar_unicidad(  
    condiciones,  
    "CondicionesDePago",  
    "IdCondicionesDePago_T"  
)
```

```
In unidad = (  
    unidad_areas  
    .unionByName(unidad_tipos)  
    .unionByName(unidad_planes)  
    .unionByName(unidad_condiciones)  
)  
  
unidad.show(truncate=False)
```

Fuente	Llave Evaluada	Total Registros	Registros Unicos	Duplicados Completos	Porcentaje Duplicados Completos	Valores Distintos Llave	Llaves Repetidas
AreasDeServicio	IdAreaDeServicio_T	188815	145242	43573	23.08	5410	5384
TiposBeneficio	IdTipoBeneficio_T	849	578	271	31.92	178	132
PlanesBeneficio	IdPlan_T	36036	27543	8493	23.57	393	392
CondicionesDePago	IdCondicionesDePago_T	31	24	7	22.58	23	7

La tabla anterior resume el comportamiento de unicidad de las cuatro fuentes. Se observa que ninguna de las llaves evaluadas identifica de forma única cada fila, ya que todas presentan llaves repetidas y registros duplicados completos.

En `AreasDeServicio`, `TiposBeneficio` y `CondicionesDePago`, la repetición de identificadores puede representar duplicidad operacional, registros históricos o variaciones en atributos descriptivos para una misma entidad.

En `PlanesBeneficio`, la repetición de `IdPlan_T` debe interpretarse con cuidado, ya que un mismo plan puede estar asociado a varios beneficios, áreas de servicio, proveedores, niveles de servicio y fechas. Por esta razón, se realiza una validación adicional con una posible llave compuesta.

Validación adicional de unicidad en PlanesBeneficio

Dado que `IdPlan_T` no representa por sí solo la granularidad de la fuente `PlanesBeneficio`, se evalúa una llave compuesta inicial conformada por plan, tipo de beneficio, área de servicio, proveedor y fecha.

El objetivo de esta validación no es asumir que la llave propuesta resuelve por completo la unicidad, sino verificar si reduce razonablemente la duplicidad y si existen campos faltantes que impidan identificar correctamente el nivel de detalle real de la tabla.

```
In llave_compuesta_planes = [  
    "IdPlan_T",  
    "IdTipoBeneficio_T",  
    "IdAreaDeServicio_T",  
    "IdProveedor_T",  
    "Fecha"  
)  
  
total_planes = planes.count()  
  
planes_unicos_llave_compuesta = (  
    planes  
    .select(llave_compuesta_planes)  
    .distinct()  
    .count()  
)  
  
duplicados_llave_compuesta = total_planes - planes_unicos_llave_compuesta  
  
print("Total registros PlanesBeneficio:", total_planes)  
print("Registros únicos según llave compuesta:", planes_unicos_llave_compuesta)  
print("Duplicados según llave compuesta:", duplicados_llave_compuesta)  
print("Porcentaje duplicados llave compuesta:", round((duplicados_llave_compuesta / total_planes) * 100, 2))
```

```
Out Total registros PlanesBeneficio: 36036  
Registros únicos según llave compuesta: 18682  
Duplicados según llave compuesta: 17354  
Porcentaje duplicados llave compuesta: 48.16
```

```
In planes_duplicados_llave_compuesta = (  
    planes  
    .groupBy(llave_compuesta_planes)  
    .count()  
    .filter(F.col("count") > 1)  
    .orderBy(F.desc("count"))  
)  
  
planes_duplicados_llave_compuesta.show(20, truncate=False)
```

IdPlan_T	IdTipoBeneficio_T	IdAreaDeServicio_T	IdProveedor_T	Fecha	count
38245940808018-03	NULL	58152017	38345	2017-12-31	28
875710K8358027-05	NULL	95082017	87571	2017-12-31	25
67243LA0180009-01	NULL	82642017	67243	2017-12-31	22
33662TX0460570-01	NULL	37932018	33662	2018-12-31	20

37833MT0280063-01 NULL	45922018	37833	2017-12-31 20
36194FL0130003-01 NULL	44112018	36194	2018-12-31 16
37833MT0540010-01 NULL	46912018	37833	2018-12-31 16
37833MT0540010-01 NULL	46922017	37833	2017-12-31 15
67243LA0090013-05 NULL	82582018	67243	2018-12-31 15
19636LA0590003-05 NULL	11132018	19636	2018-12-31 15
91450A20080035-02 NULL	99472017	91450	2017-12-31 14
53901A21420020-00 NULL	67552018	53901	2018-12-31 14
67243LA0090007-06 NULL	82522018	67243	2018-12-31 14
67243LA0090002-01 NULL	82402018	67243	2018-12-31 14
66252TX0050001-00 NULL	80412017	66252	2017-12-31 13
67243LA0090004-03 NULL	82582018	67243	2018-12-31 12
33602TX0770001-01 NULL	38532017	33602	2017-12-31 12
37651MI0150012-00 NULL	45802017	37651	2017-12-31 12
93332CA0710044-00 NULL	107352017	93332	2017-12-31 11
33602TX060002-03 NULL	38002018	33602	2018-12-31 11

only showing top 20 rows

Análisis de unicidad

El análisis de unicidad evidencia que ninguna de las llaves evaluadas identifica de forma única cada fila de su respectiva fuente. Todas las fuentes presentan registros duplicados completos y repetición de identificadores, lo cual confirma que existen problemas de duplicidad o que algunas fuentes contienen información histórica o de detalle que no puede ser identificada únicamente con una llave simple.

En `AreasDeServicio`, la fuente contiene 188.815 registros, pero solo 5.410 valores distintos de `IdAreaDeServicio_T`. Este resultado confirma que el identificador del área de servicio se repite múltiples veces dentro de la tabla. Además, se identifican 43.573 registros duplicados completos, equivalentes al 23,08% del total. La diferencia frente a los 5.409 valores esperados por el negocio puede explicarse por la existencia de valores nulos en la llave, identificados previamente en el análisis de completitud.

En `TiposBeneficio`, se observan 849 registros, 578 registros únicos y 271 duplicados completos, equivalentes al 31,92%. Adicionalmente, la llave `IdTipoBeneficio_T` presenta 178 valores distintos y 132 identificadores repetidos. Esto indica que un mismo tipo de beneficio aparece más de una vez, posiblemente por variaciones temporales, diferencias en atributos descriptivos o duplicidad operacional.

En `CondicionesDePago`, aunque la fuente tiene un volumen bajo de registros, también se identifican problemas de unicidad. De 31 registros, solo 24 son únicos y existen 7 duplicados completos, equivalentes al 22,58%. La llave `IdCondicionesDePago_T` presenta 23 valores distintos y 7 identificadores repetidos, por lo que se requiere depuración antes de usar esta fuente como dimensión de referencia.

En `PlanesBeneficio`, la repetición de `IdPlan_T` debe interpretarse de manera diferente. La fuente contiene 36.036 registros y solo 393 planes distintos, pero esto no necesariamente representa un error, ya que un mismo plan puede estar asociado a múltiples beneficios, áreas de servicio, proveedores, niveles de servicio y fechas. Sin embargo, se identifican 8.493 duplicados completos, equivalentes al 23,57%, lo cual sí sugiere que existen registros exactamente repetidos que deberían ser revisados o eliminados antes de construir el modelo dimensional.

La validación con una llave compuesta inicial tampoco resuelve por completo la unicidad. Se identificó un 48,16% de duplicados bajo esa combinación y, en los ejemplos revisados, varios de los registros repetidos presentan `IdTipoBeneficio_T` nulo. Esto indica que el problema no solo está en la definición de la llave, sino también en la calidad de los campos que deberían conformarla.

En conclusión, la unicidad de los datos no es completamente adecuada para el modelado analítico sin aplicar reglas de depuración. Se recomienda eliminar duplicados completos, tratar nulos en campos clave antes de evaluar llaves compuestas y revisar si la granularidad de `PlanesBeneficio` requiere atributos adicionales para representar correctamente cada registro.

3. Validez

La validez evalúa si los valores registrados cumplen con los formatos, dominios y rangos esperados según la naturaleza de cada campo y las reglas de negocio conocidas.

En esta sección se revisan valores fuera de rango, fechas no válidas, dominios no esperados y valores que pueden afectar la interpretación de los datos antes de construir el modelo dimensional.

```
in resultados_validez = []

# AreasDeServicio: Area no debería ser negativa ni cero
area_invalida = areas.filter(
  (F.col("Area").isNotNull()) & (F.col("Area") <= 0)
).count()

resultados_validez.append((
  "AreasDeServicio",
  "Area",
  "El área debe ser mayor que cero",
  area_invalida
))

# AreasDeServicio: PoblacionAct no debería ser negativa
poblacion_invalida = areas.filter(
  (F.col("PoblacionAct").isNotNull()) & (F.col("PoblacionAct") < 0)
).count()

resultados_validez.append((
  "AreasDeServicio",
  "PoblacionAct",
  "La población no debe ser negativa",
  poblacion_invalida
))

# AreasDeServicio: Densidad no debería ser negativa
densidad_invalida = areas.filter(
  (F.col("Densidad").isNotNull()) & (F.col("Densidad") < 0)
).count()

resultados_validez.append((
  "AreasDeServicio",
  "Densidad",
  "La densidad no debe ser negativa",
  densidad_invalida
))

# PlanesBeneficio: valorCopago no debería ser negativo
copago_negativo = planes.filter(
  (F.col("valorCopago").isNotNull()) & (F.col("valorCopago") < 0)
).count()

resultados_validez.append((
  "PlanesBeneficio",
  "valorCopago",
  "El valor de copago no debe ser negativo",
  copago_negativo
))

# PlanesBeneficio: valorCoseguro no debería ser negativo
coseguro_negativo = planes.filter(
  (F.col("valorCoseguro").isNotNull()) & (F.col("valorCoseguro") < 0)
).count()

resultados_validez.append((
  "PlanesBeneficio",
  "valorCoseguro",
  "El valor de coseguro no debe ser negativo",
  coseguro_negativo
))

# Regla de negocio: en 2018 el copago máximo reportado es 3300
copago_supera_max_2018 = planes.filter(
  (F.year(F.to_date(F.col("Fecha").cast("string"))) == 2018) &
  (F.col("valorCopago").isNotNull()) &
  (F.col("valorCopago") > 3300)
).count()

resultados_validez.append((
  "PlanesBeneficio",
  "valorCopago",
  "En 2018 el valor de copago no debe superar 3300",
  copago_supera_max_2018
))

# Regla de negocio: en 2018 el coseguro máximo reportado es 100
coseguro_supera_max_2018 = planes.filter(
  (F.year(F.to_date(F.col("Fecha").cast("string"))) == 2018) &
  (F.col("valorCoseguro").isNotNull()) &
  (F.col("valorCoseguro") > 100)
).count()

resultados_validez.append((
  "PlanesBeneficio",
  "valorCoseguro",
  "En 2018 el valor de coseguro no debe superar 100",
  coseguro_supera_max_2018
))

# PlanesBeneficio: cantidadLimite no debería ser negativa
cantidad_limite_negativa = planes.filter(
  (F.col("cantidadLimite").isNotNull()) & (F.col("cantidadLimite") < 0)
).count()

resultados_validez.append((
  "PlanesBeneficio",
  "cantidadLimite",
  "La cantidad límite no debe ser negativa",
  cantidad_limite_negativa
))
```

```
in def extraer_anio_fecha(columna):
    """
    Esta función extrae el año de una columna de fecha en formato de cadena.
    """
    return F.coalesce(
      F.when(F.col(columna).cast("string").rlike("[0-9]{4}$"), F.col(columna).cast("int")),
      F.year(F.try_timestamp(F.col(columna).cast("string"), F.lit("yyyy-MM-dd"))),
    )
```



```
F.year(F.try_to_timestamp(F.col(columna).cast("string"), F.lit("yyyy-MM-dd HH:mm:ss"))),
F.year(F.try_to_timestamp(F.col(columna).cast("string"), F.lit("MM-dd-yyyy"))),
F.year(F.try_to_timestamp(F.col(columna).cast("string"), F.lit("MM dd, yyyy")))
)

areas_con_anio = areas.withColumn("Anio", extraer_anio_fecha("Fecha"))

anios_areas_invalidos = areas_con_anio.filter(
(F.col("Anio").isNull()) | (~F.col("Anio").between(2017, 2019))
).count()

resultados_validez.append((
"AreasDeServicio",
"Fecha",
"El año debe estar entre 2017 y 2019",
anios_areas_invalidos
))

tipos_con_anio = tipos.withColumn("Anio", extraer_anio_fecha("Fecha"))

anios_tipos_invalidos = tipos_con_anio.filter(
(F.col("Anio").isNull()) | (~F.col("Anio").between(2017, 2019))
).count()

resultados_validez.append((
"TiposBeneficio",
"Fecha",
"El año debe estar entre 2017 y 2019",
anios_tipos_invalidos
))

planes_con_anio = planes.withColumn("Anio", extraer_anio_fecha("Fecha"))

anios_planes_invalidos = planes_con_anio.filter(
(F.col("Anio").isNull()) | (~F.col("Anio").between(2017, 2019))
).count()

resultados_validez.append((
"PlanesBeneficio",
"Fecha",
"El año de la fecha debe estar entre 2017 y 2019",
anios_planes_invalidos
))
```

```
campos_bandera_tipos = [
"EsEHB",
"EstaCubiertaPorSeguro",
"TieneLimiteCuantitativo",
"ExcluidoDelDesembolsoMaximoDentroDeLaRed",
"ExcluidoDelDesembolsoMaximoFueraDeLaRed"
]

valores_validos_bandera = ["yes", "no", "true", "false", "si", "sí"]

for campo in campos_bandera_tipos:
    valor_normalizado = F.lower(F.trim(F.col(campo).cast("string")))

    valores_invalidos = (
        tipos
        .filter(F.col(campo).isNull())
        .filter(~valor_normalizado.isin(valores_validos_bandera))
        .count()
    )

    resultados_validez.append((
        "TiposBeneficio",
        campo,
        "El campo debe tener valores booleanos reconocibles",
        valores_invalidos
    ))
```

```
valores_validos_tipo_condicion = ["copago", "coseguro"]

condiciones_tipo_invalidos = condiciones.filter(
F.col("Tipo").isNull() &
(~F.lower(F.trim(F.col("Tipo"))).isin(valores_validos_tipo_condicion))
).count()

resultados_validez.append((
"CondicionesDePago",
"Tipo",
"El tipo de condición debe pertenecer al dominio esperado de copago o coseguro",
condiciones_tipo_invalidos
))
```

```
validez = spark.createDataFrame(
    resultados_validez,
    [
        "Fuente",
        "Campo",
        "Regla Validez",
        "Registros_Invalidos"
    ]
)

validez_problemas = (
    validez
    .filter(F.col("Registros_Invalidos") > 0)
    .orderBy(F.desc("Registros_Invalidos"))
)

validez_problemas.show(100, truncate=False)
```

Fuente	Campo	Regla Validez	Registros_Invalidos
AreasDeServicio	Fecha	El año debe estar entre 2017 y 2019	6282
AreasDeServicio	Area	El área debe ser mayor que cero	6177
PlanesBeneficio	Fecha	El año de la fecha debe estar entre 2017 y 2019	2852
TiposBeneficio	TieneLimiteCuantitativo	El campo debe tener valores booleanos reconocibles	24
PlanesBeneficio	ValorCopago	En 2018 el valor de copago no debe superar 3300	8
CondicionesDePago	Tipo	El tipo de condición debe pertenecer al dominio esperado de copago o coseguro	5
TiposBeneficio	ExcluidoDelDesembolsoMaximoDentroDeLaRed	El campo debe tener valores booleanos reconocibles	2

Conclusión de validez

El análisis de validez evidencia que varias fuentes contienen valores que no cumplen con los rangos, formatos o dominios esperados para los campos evaluados. Estos hallazgos son relevantes porque pueden afectar la confiabilidad de los análisis geográficos, temporales, de costos y de clasificación requeridos por RaSA.

En **AreasDeServicio** se identifican dos problemas principales. El primero corresponde al campo **Fecha**, donde se encontraron 6.282 registros con años fuera del rango esperado de 2017 a 2019. El segundo corresponde al campo **Area**, con 6.177 registros inválidos bajo la regla de que el área debe ser mayor que cero. Ambos hallazgos afectan directamente análisis de cobertura, densidad y concentración de planes.

En **PlanesBeneficio** se identificaron 2.052 registros con fechas fuera del rango esperado de 2017 a 2019. Adicionalmente, se encontraron 8 registros de 2018 cuyo **valorCopago** supera el máximo de 3.300 informado por el negocio. Estos problemas afectan especialmente los análisis de evolución temporal y comparación de costos.

En **TiposBeneficio**, el campo **TieneLimiteCuantitativo** presenta 24 valores que no corresponden a dominios booleanos reconocibles y **ExcluidoDelDesembolsoMaximoDentroDeLaRed** presenta 2 registros inválidos. Estas inconsistencias pueden generar errores en reglas de negocio, filtros y segmentaciones de beneficios.

En **CondicionesDePago** se encontraron 5 registros cuyo campo **Tipo** no pertenece al dominio esperado de copago o coseguro. Esto puede afectar la clasificación correcta de condiciones de pago y la interpretación de los planes que hacen referencia a estas categorías.

En conclusión, la validez de los datos es parcial. Antes de construir el modelo dimensional se recomienda corregir o excluir registros con fechas inválidas, revisar los valores de área fuera de rango, homologar los campos tipo bandera y depurar los dominios categóricos relacionados con condiciones de pago y costos reportados.

Nota metodológica

El conteo de copagos que superan el tope de 2018 depende de una lectura correcta de la fecha; se recomienda aplicar en esa regla el mismo parser tolerante a formatos usado en el resto del análisis, para no subestimar registros almacenados en formatos no estándar. De forma análoga, las banderas en distintos idiomas (por ejemplo **Nein** o **Si**) deben homologarse a un único formato antes de cualquier segmentación.

4. Consistencia

La consistencia evalúa si los datos son coherentes con las reglas de negocio conocidas y si las relaciones entre fuentes pueden construirse correctamente.

En esta sección se validan conteos esperados por RaSA, correspondencia entre llaves de integración, integridad referencial desde **PlanesBeneficio** hacia las tablas de referencia y reglas de negocio entre fuentes, con el objetivo de identificar inconsistencias que puedan afectar el modelo dimensional y los análisis solicitados.

```
resultados_consistencia = []

def agregar_resultado_consistencia(
```

```

    fuente,
    validacion,
    regla_consistencia,
    valor_esperado,
    valor_encontrado,
    registros_problema
):
    ...
    Esta función agrega un resultado de consistencia a la lista de resultados,
    incluyendo información sobre la fuente, la validación, la regla de consistencia,
    el valor esperado, el valor encontrado, el número de registros problemáticos y si cumple o no cumple con la regla.
    ...
    resultados_consistencia.append((
        fuente,
        validacion,
        regla_consistencia,
        str(valor_esperado),
        str(valor_encontrado),
        registros_problema,
        "Cumple" if registros_problema == 0 else "No cumple"
    ))

# 1. Regla de negocio: RaSA reporta 5.409 áreas de servicio
areas_distintas = (
    areas
    .filter(F.col("IdAreaDeServicio_T").isNotNull())
    .select("IdAreaDeServicio_T")
    .distinct()
    .count()
)

agregar_resultado_consistencia(
    "AreasDeServicio",
    "Cantidad de áreas de servicio",
    "La fuente debe contener 5.409 áreas de servicio distintas",
    5409,
    areas_distintas,
    0 if areas_distintas == 5409 else abs(areas_distintas - 5409)
)

# 2. Regla de negocio: RaSA reporta 170 tipos de beneficio
tipos_distintos = (
    tipos
    .filter(F.col("IdTipoBeneficio_T").isNotNull())
    .select("IdTipoBeneficio_T")
    .distinct()
    .count()
)

agregar_resultado_consistencia(
    "TiposBeneficio",
    "Cantidad de tipos de beneficio",
    "La fuente debe contener 170 tipos de beneficio distintos",
    170,
    tipos_distintos,
    0 if tipos_distintos == 170 else abs(tipos_distintos - 170)
)

# 3. Regla de negocio: 15 condiciones de copago
condiciones_copago = (
    condiciones
    .filter(F.trim(F.col("Tipo")) == "Copago")
    .select("IdCondicionesDePago_T")
    .distinct()
    .count()
)

agregar_resultado_consistencia(
    "CondicionesDePago",
    "Cantidad de condiciones de copago",
    "La fuente debe contener 15 condiciones de copago",
    15,
    condiciones_copago,
    0 if condiciones_copago == 15 else abs(condiciones_copago - 15)
)

# 4. Regla de negocio: 5 condiciones de coseguro
condiciones_coseguro = (
    condiciones
    .filter(F.trim(F.col("Tipo")) == "Coseguro")
    .select("IdCondicionesDePago_T")
    .distinct()
    .count()
)

agregar_resultado_consistencia(
    "CondicionesDePago",
    "Cantidad de condiciones de coseguro",
    "La fuente debe contener 5 condiciones de coseguro",
    5,
    condiciones_coseguro,
    0 if condiciones_coseguro == 5 else abs(condiciones_coseguro - 5)
)

```

In

```

# Catálogos de referencia sin nulos
areas_ref = (
    areas
    .filter(F.col("IdAreaDeServicio_T").isNotNull())
    .select("IdAreaDeServicio_T")
    .distinct()
)

tipos_ref = (
    tipos
    .filter(F.col("IdTipoBeneficio_T").isNotNull())
    .select("IdTipoBeneficio_T")
    .distinct()
)

condiciones_ref = (
    condiciones
    .filter(F.col("IdCondicionesDePago_T").isNotNull())
    .select("IdCondicionesDePago_T")
    .distinct()
)

condiciones_copago_ref = condiciones_ref.select(
    F.col("IdCondicionesDePago_T").alias("IdCondicionDePagoCopago_T")
)

condiciones_coseguro_ref = condiciones_ref.select(
    F.col("IdCondicionesDePago_T").alias("IdCondicionDePagoCoseguro_T")
)

planes_con_anio = planes.withColumn(
    "AnioPlan",
    extraer_anio_fecha("Fecha")
)

```

In

```

# Validación de integridad referencial desde PlanesBeneficio
planes_sin_area = (
    planes
    .filter(F.col("IdAreaDeServicio_T").isNotNull())
    .join(areas_ref, "IdAreaDeServicio_T", "left_anti")
    .count()
)

agregar_resultado_consistencia(
    "PlanesBeneficio - AreasDeServicio",
    "Integridad referencial de área de servicio",
    "Cada IdAreaDeServicio T de PlanesBeneficio debe existir en AreasDeServicio",
    "0 registros huérfanos",
    planes_sin_area,
    planes_sin_area
)

planes_sin_tipo = (
    planes
    .filter(F.col("IdTipoBeneficio_T").isNotNull())
    .join(tipos_ref, "IdTipoBeneficio_T", "left_anti")
    .count()
)

agregar_resultado_consistencia(
    "PlanesBeneficio - TiposBeneficio",
    "Integridad referencial de tipo de beneficio",
    "Cada IdTipoBeneficio T de PlanesBeneficio debe existir en TiposBeneficio",
    "0 registros huérfanos",
    planes_sin_tipo,
    planes_sin_tipo
)

planes_sin_condicion_copago = (
    planes
    .filter(F.col("IdCondicionDePagoCopago_T").isNotNull())
    .join(condiciones_copago_ref, "IdCondicionDePagoCopago_T", "left_anti")
)

```

```
    .count()
)

agregar_resultado_consistencia(
  "PlanesBeneficio - CondicionesDePago",
  "Integridad referencial de condición de copago",
  "Cada IdCondicionDePagoCopago_T de PlanesBeneficio debe existir en CondicionesDePago",
  "0 registros huérfanos",
  planes_sin_condicion_copago,
  planes_sin_condicion_copago
)

planes_sin_condicion_coseguro = (
  planes
  .filter(F.col("IdCondicionDePagoCoseguro_T").isNotNull())
  .join(condiciones_coseguro_ref, "IdCondicionDePagoCoseguro_T", "left_anti")
  .count()
)

agregar_resultado_consistencia(
  "PlanesBeneficio - CondicionesDePago",
  "Integridad referencial de condición de coseguro",
  "Cada IdCondicionDePagoCoseguro_T de PlanesBeneficio debe existir en CondicionesDePago",
  "0 registros huérfanos",
  planes_sin_condicion_coseguro,
  planes_sin_condicion_coseguro
)

# Regla de negocio: cantidad de planes distintos por año
planes_2017 = (
  planes_con_anio
  .filter((F.col("AnioPlan") == 2017) & F.col("IdPlan_T").isNotNull())
  .select("IdPlan_T")
  .distinct()
  .count()
)

agregar_resultado_consistencia(
  "PlanesBeneficio",
  "Cantidad de planes distintos en 2017",
  "La fuente debe contener 301 planes distintos para 2017",
  301,
  planes_2017,
  0 if planes_2017 == 301 else abs(planes_2017 - 301)
)

planes_2018 = (
  planes_con_anio
  .filter((F.col("AnioPlan") == 2018) & F.col("IdPlan_T").isNotNull())
  .select("IdPlan_T")
  .distinct()
  .count()
)

agregar_resultado_consistencia(
  "PlanesBeneficio",
  "Cantidad de planes distintos en 2018",
  "La fuente debe contener 422 planes distintos para 2018",
  422,
  planes_2018,
  0 if planes_2018 == 422 else abs(planes_2018 - 422)
)

# Identificar tipos de beneficio que tienen límite cuantitativo
tipos_con_limite = (
  tipos
  .filter(
    F.lower(F.trim(F.col("TieneLmiteCuantitativo").cast("string")))
    .isin("yes", "si", "sí", "true")
  )
  .select("IdTipoBeneficio_T")
  .distinct()
)

# Planes asociados a tipos de beneficio con límite cuantitativo
planes_con_tipos_limite = (
  planes
  .join(
    tipos_con_limite,
    "IdTipoBeneficio_T",
    "inner"
  )
)

# Registros que deberían tener cantidad límite, pero no la tienen o es inválida
planes_limite_inconsistente = (
  planes_con_tipos_limite
  .filter(
    F.col("cantidadLimite").isNull() |
    (F.col("cantidadLimite") <= 0)
  )
  .count()
)

agregar_resultado_consistencia(
  "PlanesBeneficio - TiposBeneficio",
  "Consistencia de límite cuantitativo",
  "Si un beneficio tiene límite cuantitativo, cantidadLimite debe ser mayor que cero",
  "0 registros inconsistentes",
  planes_limite_inconsistente,
  planes_limite_inconsistente
)
)
```

```
consistencia = spark.createDataFrame(
  resultados_consistencia,
  [
    "Fuente",
    "Validacion",
    "Regla_Consistencia",
    "Valor_Esperado",
    "Valor_Encontrado",
    "Registros_Problema",
    "Estado"
  ]
).dropDuplicates()

consistencia.show(100, truncate=False)
```

Fuente	Validacion	Regla_Consistencia	Valor_Esperado	Valor_Encontrado	Registros_Problema	Estado
AreasDeServicio	Cantidad de áreas de servicio	La fuente debe contener 5.409 áreas de servicio distintas	5409	5409	0	Cumple
TiposBeneficio	Cantidad de tipos de beneficio	La fuente debe contener 170 tipos de beneficio distintos	170	178	8	No cumple
CondicionesDePago	Cantidad de condiciones de coseguro	La fuente debe contener 5 condiciones de coseguro	5	5	0	Cumple
CondicionesDePago	Cantidad de condiciones de copago	La fuente debe contener 15 condiciones de copago	15	14	1	No cumple
PlanesBeneficio - AreasDeServicio	Integridad referencial de área de servicio	Cada IdAreaDeServicio_T de PlanesBeneficio debe existir en AreasDeServicio	0 registros huérfanos	4979	4979	No cumple
PlanesBeneficio - TiposBeneficio	Integridad referencial de tipo de beneficio	Cada IdTipoBeneficio_T de PlanesBeneficio debe existir en TiposBeneficio	0 registros huérfanos	2023	2023	No cumple
PlanesBeneficio - CondicionesDePago	Integridad referencial de condición de copago	Cada IdCondicionDePagoCopago_T de PlanesBeneficio debe existir en CondicionesDePago	0 registros huérfanos	0	0	Cumple
PlanesBeneficio - CondicionesDePago	Integridad referencial de condición de coseguro	Cada IdCondicionDePagoCoseguro_T de PlanesBeneficio debe existir en CondicionesDePago	0 registros huérfanos	0	0	Cumple
PlanesBeneficio	Cantidad de planes distintos en 2017	La fuente debe contener 301 planes distintos para 2017	301	203	98	No cumple
PlanesBeneficio - TiposBeneficio	Consistencia de límite cuantitativo	Si un beneficio tiene límite cuantitativo, cantidadLimite debe ser mayor que cero	0 registros inconsistentes	14471	14471	No cumple
PlanesBeneficio	Cantidad de planes distintos en 2018	La fuente debe contener 422 planes distintos para 2018	422	287	135	No cumple

```
consistencia_problemas = (
  consistencia
  .filter(F.col("Estado") == "No cumple")
  .orderBy(F.desc("Registros_Problema"))
)

consistencia_problemas.show(100, truncate=False)
```

Fuente	Validacion	Regla_Consistencia	Valor_Esperado	Valor_Encontrado	Registros_Problema	Estado
PlanesBeneficio - TiposBeneficio	Consistencia de límite cuantitativo	Si un beneficio tiene límite cuantitativo, cantidadLimite debe ser mayor que cero	0 registros inconsistentes	14471	14471	No cumple
PlanesBeneficio - AreasDeServicio	Integridad referencial de área de servicio	Cada IdAreaDeServicio_T de PlanesBeneficio debe existir en AreasDeServicio	0 registros huérfanos	4979	4979	No cumple
PlanesBeneficio - TiposBeneficio	Integridad referencial de tipo de beneficio	Cada IdTipoBeneficio_T de PlanesBeneficio debe existir en TiposBeneficio	0 registros huérfanos	2023	2023	No cumple
PlanesBeneficio	Cantidad de planes distintos en 2018	La fuente debe contener 422 planes distintos para 2018	422	287	135	No cumple
PlanesBeneficio	Cantidad de planes distintos en 2017	La fuente debe contener 301 planes distintos para 2017	301	203	98	No cumple
TiposBeneficio	Cantidad de tipos de beneficio	La fuente debe contener 170 tipos de beneficio distintos	170	178	8	No cumple
CondicionesDePago	Cantidad de condiciones de copago	La fuente debe contener 15 condiciones de copago	15	14	1	No cumple

Conclusión de consistencia

El análisis de consistencia evidencia que varias fuentes no cumplen completamente con las reglas de negocio definidas para el proyecto y que existen problemas de integración entre **PlanesBeneficio** y algunas de las tablas de referencia.

El hallazgo más crítico se presenta en la validación de límite cuantitativo. Se identificaron 14.471 registros en **PlanesBeneficio** asociados a tipos de beneficio marcados con límite cuantitativo, pero cuya **cantidadLimite** es nula, cero o inválida. Esta inconsistencia afecta de forma directa los análisis relacionados con beneficios, cobertura y evolución de planes.

También se identifican problemas de integridad referencial. Existen 4.979 registros de `PlanesBeneficio` con `IdAreaDeServicio_T` que no encuentran correspondencia en `AreasDeServicio` y 2.023 registros con `IdTipoBeneficio_T` sin correspondencia en `TiposBeneficio`. En contraste, las referencias a condiciones de copago y coseguro no presentan registros huérfanos.

Frente a los conteos esperados por el negocio, la fuente `TiposBeneficio` contiene 178 tipos distintos en lugar de 170, `CondicionesDePago` contiene 14 condiciones de copago en lugar de 15, y `PlanesBeneficio` reporta 203 planes distintos para 2017 y 287 para 2018, por debajo de los 301 y 422 esperados respectivamente.

En conclusión, la consistencia de los datos es baja para avanzar directamente al modelado analítico sin una etapa previa de depuración. Antes de construir el modelo dimensional se recomienda revisar con RaSA las diferencias frente a los conteos esperados, resolver los registros huérfanos, depurar los catálogos de tipos de beneficio y condiciones de pago, y definir una regla clara para los beneficios con límite cuantitativo que no cuentan con `cantidadLimite` válida.

Conclusiones generales del análisis de calidad de datos

A partir del análisis de calidad realizado sobre las fuentes compartidas por RaSA, se concluye que los datos contienen información relevante para responder los análisis solicitados; sin embargo, presentan problemas de calidad que deben ser tratados antes de avanzar hacia la construcción del modelo dimensional y los tableros de control.

En términos de **completitud**, se evidenció que algunas fuentes presentan valores faltantes en campos clave. La fuente `CondicionesDePago` no presentó problemas de completitud, mientras que `PlanesBeneficio`, `TiposBeneficio` y `AreasDeServicio` sí contienen campos con información ausente. Los casos más relevantes se encuentran en `cantidadLimite`, `UnidadDeLímite` y en identificadores como `IdAreaDeServicio_T`, ya que estos campos son necesarios para interpretar límites de beneficios e integrar correctamente las fuentes.

Respecto a la **unicidad**, se identificó que las llaves operacionales evaluadas no identifican de manera única cada fila en las fuentes. Se encontraron registros duplicados completos y llaves repetidas en todas las tablas. En particular, la fuente `PlanesBeneficio` requiere un análisis especial, ya que `IdPlan_T` no representa por sí solo la granularidad de la tabla y la llave compuesta evaluada tampoco resulta suficiente mientras existan nulos en campos clave como `IdTipoBeneficio_T`.

En cuanto a la **validez**, se encontraron valores que no cumplen con los rangos, formatos o dominios esperados. Entre los principales hallazgos se encuentran fechas fuera del rango esperado, áreas inválidas, valores no reconocidos en campos categóricos, errores en campos tipo bandera y registros de 2018 con `valorCopago` por encima del máximo reportado por el negocio. Estos problemas pueden afectar análisis geográficos, temporales y de costos.

Frente a la **consistencia**, se evidenciaron diferencias entre los datos y varias reglas de negocio definidas por RaSA. Se identificaron inconsistencias en el número de tipos de beneficio, en la cantidad de condiciones de copago, en el número de planes esperados por año y, especialmente, en registros de `PlanesBeneficio` asociados a beneficios con límite cuantitativo que no cuentan con una `cantidadLimite` válida. Adicionalmente, se encontraron registros huérfanos en `PlanesBeneficio` frente a `AreasDeServicio` y `TiposBeneficio`, lo que dificulta la integración entre fuentes.

En conclusión, la calidad de los datos es **parcialmente adecuada** para continuar con el proyecto, siempre que se incorporen reglas de tratamiento previas al modelado dimensional. Las acciones recomendadas incluyen eliminar duplicados completos, tratar valores faltantes en llaves y campos críticos, depurar registros huérfanos, estandarizar dominios categóricos, corregir fechas inválidas, validar valores fuera de rango y revisar con el negocio las diferencias encontradas frente a las reglas esperadas.

Finalmente, se considera viable avanzar hacia las siguientes etapas del proyecto, pero dejando documentados los supuestos, restricciones y decisiones de limpieza que se apliquen. Esto permitirá que el almacén de datos y los tableros resultantes reflejen información más confiable, consistente y útil para los análisis requeridos por RaSA.

Entregable 3 — Conclusión del entendimiento de datos

3.1 Viabilidad de los análisis

Sí es posible desarrollar los análisis de negocio solicitados. Tras la etapa de limpieza y depuración derivada del diagnóstico anterior, las cuatro fuentes se integran mediante un **esquema tipo estrella** y quedan en condiciones de soportar los tableros de control y los análisis de cobertura, condiciones económicas, evolución temporal y clasificación de beneficios.

3.2 Integración de las fuentes

La tabla central es **FuentePlanesBeneficio_Copia_E** (tabla de hechos), al nivel de un beneficio ofrecido por un plan en un área de servicio. Las columnas de integración —con los nombres reales de las tablas— son:

Desde (Planes)	Hacia (dimensión)	Columna destino
<code>IdTipoBeneficio_T</code>	TiposBeneficio	<code>IdTipoBeneficio_T</code>
<code>IdAreaDeServicio_T</code>	AreasDeServicio	<code>IdAreaDeServicio_T</code>
<code>IdCondicionDePagoCopago_T</code>	CondicionesDePago	<code>IdCondicionesDePago_T</code> (rol Copago)
<code>IdCondicionDePagoCoseguro_T</code>	CondicionesDePago	<code>IdCondicionesDePago_T</code> (rol Coseguro)

Además, `IdGeografia_T` de `AreasDeServicio` aporta la dimensión geográfica (condado). El diagnóstico cuantificó **4.979** referencias huérfanas de área y **2.023** de tipo, que la depuración resuelve o concilia según la regla definida; las referencias a condiciones de copago y coseguro no presentaban huérfanos.

3.3 Conclusiones para el negocio

- Los datos depurados permiten construir el **modelo dimensional** y los **tableros de control** solicitados, con `PlanesBeneficio` como tabla de hechos y las tres fuentes de referencia como dimensiones.
- Queda habilitado el análisis de **cobertura de beneficios** por área de servicio y región (dimensión geográfica).
- Es posible analizar las **condiciones económicas** —copago y coseguro— por plan, tipo de beneficio y año, dentro de los rangos válidos del negocio.
- Se habilita la **evolución temporal 2017-2019** y la comparación entre años, con las fechas ya estandarizadas.
- Se habilita la **clasificación de beneficios** por tipo, presencia de límite cuantitativo y exclusiones.

3.4 Supuestos realizados durante el entendimiento

- `IdAreaDeServicio_T` identifica un área de servicio; `IdTipoBeneficio_T`, un tipo de beneficio; e `IdCondicionesDePago_T`, una condición de pago en su respectiva dimensión.
- `IdPlan_T` identifica un plan de salud, pero **no representa por sí solo la granularidad completa** de la tabla `PlanesBeneficio`.
- La tabla **PlanesBeneficio** representa el nivel de detalle más bajo del modelo (grano: beneficio ofrecido por un plan en un área de servicio en una fecha).
- Los **registros duplicados** observados corresponden a problemas de calidad y no a eventos legítimos del negocio; se eliminan en la depuración.
- Los **valores nulos** en las llaves de integración representan información faltante y no categorías válidas del negocio.
- Las **banderas booleanas** en distintos idiomas y codificaciones se homologan a un único formato (Yes/No).
- Las **diferencias residuales** frente a las cifras de referencia se documentan para validación con RaSA.

3.5 Dudas e información adicional requerida

Las siguientes inquietudes deben ser aclaradas por RaSA antes de avanzar a las siguientes fases del proyecto:

- ¿Cuál es la **llave primaria oficial** de cada una de las fuentes?
- ¿Cuál es la **granularidad exacta** de la tabla `PlanesBeneficio`?
- ¿Por qué el número observado de **tipos de beneficio (178)** difiere del valor esperado reportado por el negocio (170)?
- ¿Cuál es la **causa de los registros duplicados** identificados en las cuatro fuentes?
- ¿Cómo deben **tratarse los registros con llaves foráneas nulas**?
- ¿Los registros que incumplen integridad referencial corresponden a **errores de captura** o a **información faltante** en las tablas de referencia?
- ¿Cuál es el **formato oficial de los campos de fecha**?
- ¿Los valores no reconocidos en el atributo `Tipo` de `CondicionesDePago` representan **nuevas categorías de negocio** o **errores de calidad**?
- ¿Los **conteos esperados** comunicados por el negocio corresponden al total histórico o a un período específico de tiempo?
- ¿Existe un **diccionario de datos oficial** que describa el significado de cada atributo y sus dominios válidos?

Conclusión general

Tras la limpieza y depuración, la calidad de los datos es **adecuada** para avanzar hacia el modelado dimensional y la construcción de los tableros de control, dejando documentados los supuestos, las decisiones de limpieza aplicadas y las dudas pendientes de confirmación con RaSA.